

AI Chat 全栈项目

架构文档 & 源码详解

Python FastAPI · MySQL · Nginx · OpenCode

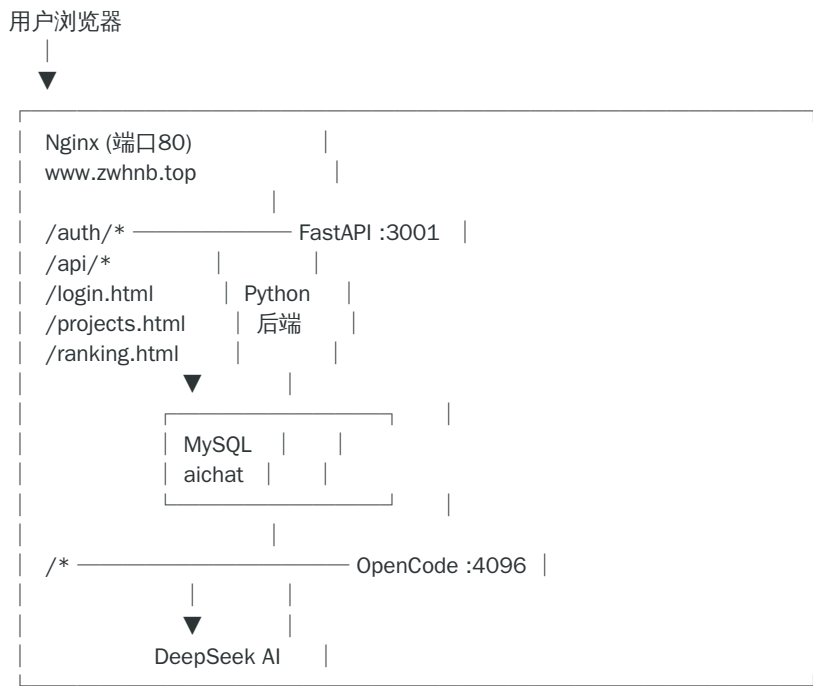
2026年7月 · 版本 2.0

目 录

01	系统架构概览	
02	技术栈清单	
03	目录结构	
04	配置中心 — config.py	
05	数据库层 — database.py	
06	认证安全 — auth.py	
07	数据模型 — models.py	
08	认证路由 — routes_auth.py	
09	项目路由 — routes_projects.py	
10	文件下载 — routes_files.py	
11	统计排名 — routes_stats.py	
12	技能插件系统 — skills/	
13	入口文件 — main.py	
14	API 接口速查表	
15	安全特性清单	

01 系统架构概览

整个系统以 Nginx 为统一入口，根据请求路径分发到不同的后端服务。



数据流说明：

- 用户浏览器 → Nginx → FastAPI (3001) 处理认证和业务逻辑
- FastAPI → OpenCode (4096) 获取 AI 会话和 Token 统计
- FastAPI → MySQL 存储用户、项目归属、统计数据
- OpenCode → DeepSeek API 调用云端 AI 模型



02 技术栈清单

层级	技术	说明
后端框架	FastAPI 0.139	异步 Web 框架，自动生成 Swagger
ASGI 服务器	Uvicorn 0.51	高性能异步，支持 uvloop
数据库驱动	aiomysql	MySQL 异步连接池
数据校验	Pydantic 2	请求/响应自动类型校验
认证	PyJWT + bcrypt	JWT 令牌 + 密码哈希
HTTP 客户端	httpx	异步请求 OpenCode API
反向代理	Nginx	统一入口、Cookie 鉴权
前端	HTML/CSS/JS	原生实现，零框架依赖



03 目录结构

server/	
—— run.sh	← systemd 启动脚本
—— .venv/	← Python 虚拟环境
└—— python_app/	← ★ 新版 Python 后端
—— main.py	← 应用入口 & 路由注册
—— config.py	← 集中配置管理
—— database.py	← 异步数据库连接池
—— auth.py	← JWT 认证 + 频率限制
—— models.py	← Pydantic 数据模型
—— routes_auth.py	← 登录/注册/忘记密码
—— routes_projects.py	← 项目管理 + 文件搜索
—— routes_files.py	← 文件下载 API
—— routes_stats.py	← Token 排名统计
└—— skills/	← 技能插件系统
—— base.py	← 技能抽象基类
—— registry.py	← 技能注册表
└—— hello.py	← 示例技能



04 配置中心 — config.py

63行 — 集中管理所有运行配置

```
"""
=====
AI Chat 后端配置模块
所有运行配置集中管理，便于后期添加快捷修改
=====
"""

import os
from pathlib import Path

# 获取环境变量，未设置则使用默认值（生产环境必须设置环境变量）
JWT_SECRET: str = os.getenv("JWT_SECRET", "")
JWT_EXPIRE_HOURS: int = 48 # Token 过期时间（小时）

# MySQL 连接配置（强制使用环境变量以保证安全）
DB_CONFIG: dict = {
    "host": os.getenv("DB_HOST", "localhost"),
    "user": os.getenv("DB_USER", "root"),
    "password": os.getenv("DB_PASSWORD", ""),
    "db": os.getenv("DB_NAME", "aichat"),
    "charset": "utf8mb4",
    "minsize": 2, # 连接池最小连接数
    "maxsize": 10, # 连接池最大连接数
}

# OpenCode 服务地址（用于获取会话和创建项目）
OPENCODE_BASE_URL: str = os.getenv("OPENCODE_URL", "http://localhost:4096")

# 文件管理配置
MAX_FILES: int = 100 # 每个项目最多保留文件数
FILE_DIR: Path = Path("/tmp/opencode/generated_files") # 生成文件存储目录
FILE_DIR.mkdir(parents=True, exist_ok=True) # 确保目录存在

# 文件下载白名单（只有这些目录下的文件允许被下载）
ALLOWED_DIRS: list[Path] = [
    Path("/tmp/opencode/generated_files"),
    Path("/tmp/opencode"),
    Path("/home/ubuntu"),
]

# 文件搜索目录（生成文件的来源目录）
SEARCH_DIRS: list[str] = ["/tmp/opencode", "/home/ubuntu"]

# 文件搜索排除规则（防止源代码文件被收录）
FIND_EXCLUDES: list[str] = [
    "*/node_modules/*", "*/.git/*", "*/ai-chat/*", "*/.cache/*",
    ... (省略后续代码)
```

05 数据库层 — database.py

77行 — aiomysql 异步连接池

```
"""
=====
数据库模块
使用 aiomysql 异步连接池（适配 FastAPI 异步运行）
=====
"""

import aiomysql
from .config import DB_CONFIG

# 全局连接池（启动时创建）
_pool: aiomysql.Pool | None = None

async def init_db() -> None:
    """
    初始化数据库连接池。
    在 FastAPI 启动事件中调用。
    """
    global _pool
    _pool = await aiomysql.create_pool(**DB_CONFIG)

async def close_db() -> None:
    """
    关闭数据库连接池。
    在 FastAPI 关闭事件中调用，确保优雅退出。
    """
    global _pool
    if _pool:
        _pool.close()
        await _pool.wait_closed()

async def get_pool() -> aiomysql.Pool:
    """
    获取连接池实例。如果未初始化则抛出异常。
    供所有数据访问模块调用。
    """
    if _pool is None:
        raise RuntimeError("数据库未初始化！请调用 init_db()")
    return _pool

async def execute(sql: str, params: tuple | list | None = None) -> list[dict]:
    ... (省略后续代码)
```

06 认证安全 — auth.py

129行 — JWT + 频率限制 + 认证中间件

```
"""
=====
认证与安全模块
JWT 令牌签发/验证 + 频率限制 + 认证中间件
=====
"""

import time
import jwt
from fastapi import Request, HTTPException
from fastapi.responses import JSONResponse
from .config import JWT_SECRET, JWT_EXPIRE_HOURS, RATE_LIMITS

# ----- JWT 令牌工具 -----

def create_token(user_id: int, username: str) -> str:
    """
    签发 JWT 令牌。
    载荷中存入用户ID和用户名，后端路由可从中提取用户身份。
    """
    payload = {
        "id": user_id,
        "username": username,
        "iat": int(time.time()),          # 签发时间
        "exp": int(time.time()) + JWT_EXPIRE_HOURS * 3600, # 过期时间
    }
    return jwt.encode(payload, JWT_SECRET, algorithm="HS256")

def verify_token(token: str) -> dict:
    """
    验证 JWT 令牌，返回解码后的载荷（包含 id, username）。
    令牌无效或过期时抛出异常。
    """
    return jwt.decode(token, JWT_SECRET, algorithms=["HS256"])

# ----- 内存频率限制 -----

# 存储结构: { "login:192.168.1.1": [timestamp1, timestamp2, ...], ... }
_rate_store: dict[str, list[float]] = {}

def rate_limit(key: str, max_req: int = 10, window_secs: float = 60.0) -> bool:
    """
    ... (省略后续代码)
```


07 数据模型 — models.py

156行 — Pydantic 请求/响应校验模型

```
"""
=====
Pydantic 数据模型
定义所有 API 请求参数和返回数据的类型结构
FastAPI 会自动根据这些模型校验数据、生成 API 文档
=====
"""

from __future__ import annotations
from pydantic import BaseModel, Field, field_validator, ConfigDict
from typing import Optional
import re

# ----- 认证相关模型 -----

class LoginRequest(BaseModel):
    """用户登录请求"""
    username: str = Field(..., min_length=1, max_length=50, examples=["admin"])
    password: str = Field(..., min_length=1, max_length=100, examples=["your_password"])

class RegisterRequest(BaseModel):
    """用户注册请求"""
    username: str = Field(..., min_length=1, max_length=50)
    password: str = Field(..., min_length=1, max_length=100)
    email: str = Field(..., max_length=200)

    @field_validator("username")
    @classmethod
    def validate_username(cls, v: str) -> str:
        if len(v) < 3 or len(v) > 20:
            raise ValueError("用户名3-20位")
        if not re.match(r'^[a-zA-Z0-9_]+$', v):
            raise ValueError("用户名只能包含字母、数字和下划线")
        return v

    @field_validator("password")
    @classmethod
    def validate_password(cls, v: str) -> str:
        if len(v) < 8:
            raise ValueError("密码至少8位")
        if not re.search(r'[a-zA-Z]', v) or not re.search(r'[0-9]', v):
            raise ValueError("密码必须包含字母和数字")
        return v

    @field_validator("email")
    ... (省略后续代码)
```

08 认证路由 — routes_auth.py

116行 — 登录 / 注册 / 忘记密码

```
"""
=====
认证路由
登录 / 注册 / 忘记密码
=====
"""

import bcrypt
from fastapi import APIRouter, Request, HTTPException
from fastapi.responses import JSONResponse

from .database import execute, execute_one, execute_write
from .auth import create_token, get_current_user
from .models import (
    LoginRequest, RegisterRequest, ForgotPasswordRequest,
    LoginResponse, UserInfo, ErrorResponse,
)

router = APIRouter(prefix="/api/auth", tags=["认证"])

@router.post("/login", response_model=LoginResponse, responses={401: {"model": ErrorResponse}})
async def login(req: LoginRequest, request: Request):
    """
    用户登录。

    流程：
    1. 查数据库校验用户名密码 -> bcrypt 比对哈希
    2. 签发 48 小时 JWT 令牌
    3. 更新最后登录时间
    4. 返回 token 和用户信息（不含密码）

    安全：频率限制由中间件控制，同一 IP 每分钟最多 10 次
    """
    user = await execute_one(
        "SELECT * FROM users WHERE username = %s AND status = %s",
        (req.username, "active"),
    )
    if not user:
        raise HTTPException(status_code=401, detail={"success": False, "message": "用户名或密码错误"})

    # bcrypt 比对：用户输入的密码 vs 数据库中的哈希值
    if not bcrypt.checkpw(req.password.encode(), user["password"].encode()):
        raise HTTPException(status_code=401, detail={"success": False, "message": "用户名或密码错误"})

    # 签发 token（载荷：用户ID + 用户名）
    ... (省略后续代码)
```

09 项目路由 — routes_projects.py

173行 — 项目CRUD + find搜索文件

```
"""
=====
项目路由
项目列表 / 创建项目 / 项目文件列表
=====
"""

import httpx
from fastapi import APIRouter, HTTPException, Depends

from .database import execute, execute_write
from .auth import get_current_user
from .config import OPENCODE_BASE_URL, MAX_FILES, FILE_DIR, SEARCH_DIRS, NEWER_REF, FIND_EXCLUD
from .models import (
    ProjectItem, ProjectListResponse, ProjectCreateResponse,
    FileItem, FileListResponse,
)

router = APIRouter(prefix="/api", tags=["项目"])

@router.get("/projects", response_model=ProjectListResponse)
async def list_projects(user: dict = Depends(get_current_user)):
    """
    获取当前登录用户的项目列表。
    流程：获取 opencode sessions -> 过滤出属于当前用户的 -> 返回
    """
    current_user = user["username"]

    async with httpx.AsyncClient() as client:
        resp = await client.get(f"{OPENCODE_BASE_URL}/api/session")
        sessions = resp.json().get("data", [])

    rows = await execute(
        "SELECT session_id FROM user_sessions WHERE username = %s",
        (current_user,)
    )
    user_session_ids = {r["session_id"] for r in rows}

    projects = []
    for s in sessions:
        if s["id"] in user_session_ids:
            projects.append(ProjectItem(
                id=s["id"],
                title=s.get("title") or "未命名",
                dir=s.get("location", {}).get("directory", "/home"),
            ))
    ... (省略后续代码)
```

10 文件下载 — routes_files.py

139行 — 单文件/打包下载 + 安全白名单

```
"""
=====
  文件下载路由
  单文件下载 / 按项目打包下载 / 一键下载最新文件
=====
"""

import subprocess
import shutil
from pathlib import Path
from fastapi import APIRouter, HTTPException, Depends, Query
from fastapi.responses import FileResponse
from starlette.background import BackgroundTask

from .auth import get_current_user
from .config import ALLOWED_DIRS, FILE_DIR, OPENCODE_BASE_URL, MAX_FILES

router = APIRouter(prefix="/api", tags=["下载"])

def _is_path_allowed(target: str) -> bool:
    """检查文件路径是否在白名单目录内（防路径遍历攻击）"""
    resolved = Path(target).resolve()
    return any(resolved.is_relative_to(d) for d in ALLOWED_DIRS)

def _cleanup_file(filepath: Path) -> None:
    """后台清理临时文件"""
    try:
        filepath.unlink(missing_ok=True)
    except Exception:
        pass

@router.get("/download/file")
async def download_file(
    path: str = Query(..., description="文件绝对路径"),
    user: dict = Depends(get_current_user),
):
    """
    下载单个文件。

    安全策略：
    1. 路径必须在白名单目录内（/tmp/opencode, /home/ubuntu）
    2. 只有文件可被下载（目录不可）
    3. 使用 FileResponse 流式传输，浏览器触发下载
    ... (省略后续代码)
    """
```

11 统计排名 — routes_stats.py

164行 — Token排名 + 隐私遮蔽

```
"""
=====
统计排名路由
项目 Token 排名 / 用户 Token 排名
=====
"""

from datetime import datetime, timezone, timedelta
import httpx
from fastapi import APIRouter, Depends

from .database import execute, execute_write
from .auth import get_current_user
from .config import OPENCODE_BASE_URL, ONLINE_THRESHOLD_MS, ADMIN_USERNAME

router = APIRouter(prefix="/api/stats", tags=["统计"])

def _mask(s: str) -> str:
    """隐私遮蔽：只显示首尾字符，中间用*代替"""
    if not s:
        return ""
    if len(s) <= 2:
        return s[0] + "*"
    hidden_len = min(len(s) - 2, 4)
    return s[0] + "*" * hidden_len + s[-1]

@router.get("/ranking")
async def project_ranking(user: dict = Depends(get_current_user)):
    """
    项目 Token 排名。

    从 opencode 获取所有会话，按标题聚合 Token 用量，
    按 total 降序排列返回前 50 名。
    """
    async with httpx.AsyncClient() as client:
        resp = await client.get(f"{OPENCODE_BASE_URL}/api/session")
        sessions = resp.json().get("data", [])

    # 按标题聚合
    stats: dict[str, dict] = {}
    for s in sessions:
        key = s.get("title") or "未命名"
        if key not in stats:
            stats[key] = {
... (省略后续代码)
```

12 技能插件系统 — skills/

技能插件是系统的可扩展机制。每个技能是一个 Python 类，继承 BaseSkill，在 on_register() 中注册路由或事件。启动时由 SkillRegistry 统一安装。

12.1 基类 — base.py (24行)

定义技能的抽象接口，所有自定义技能必须继承此类。

```
"""
=====

技能插件系统基类
定义技能的抽象接口，供具体技能实现继承。

后期扩展方式：
1. 在 skills/ 目录下新建 Python 文件（如 ppt_skill.py）
2. 继承 BaseSkill 并实现 on_register() 方法
3. 在 skills/registry.py 中注册
=====
"""

from __future__ import annotations
from abc import ABC, abstractmethod
from typing import Any
from fastapi import FastAPI

class BaseSkill(ABC):
    """
    技能基类。
    ... (省略后续代码)
    """
```

12.2 注册表 — registry.py (46行)

单例模式，管理所有技能。在 startup 事件中批量安装到 FastAPI app。

```
"""
=====

技能注册表
管理所有技能的加载和注册。

使用方法:
from .skill_registry import registry

# 加载技能
registry.register(MySkill())

# 在 main.py 启动时安装所有技能
await registry.install(app)
=====
"""

from __future__ import annotations
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from fastapi import FastAPI
```

```
from .base import BaseSkill

class SkillRegistry:
    """
    ... (省略后续代码)
```

12.3 示例 — hello.py (20行)

演示如何创建一个技能：定义路由 → 注册到 app。

```
"""
=====
Skill: Hello World 示例技能
演示如何创建一个技能插件

后期可以新建 skill_ppt.py, skill_agent.py 等
=====
"""

from fastapi import FastAPI, APIRouter
from .base import BaseSkill

class HelloSkill(BaseSkill):
    """示例技能：演示技能插件机制"""
    name = "hello_world"
    description = "Hello World 示例技能"

    async def on_register(self, app: FastAPI) -> None:
        router = APIRouter(prefix="/api/skills/hello", tags=["技能示例"])

    ... (省略后续代码)
```

★ 添加新技能只需3步：

1. 在 skills/ 目录新建文件 (如 ppt_skill.py)
 2. 继承 BaseSkill，实现 on_register(app) 方法
 3. 在 main.py 的 lifespan 中注册: registry.register(MySkill())
-

13 入口文件 — main.py

155行 — FastAPI 应用创建、生命周期管理、路由注册、全局错误处理。

```
"""
=====
AI Chat 后端主入口
技术栈: Python 3.12 + FastAPI + aiomysql + JWT + bcrypt

架构层次：
main.py      → 启动入口，组装所有路由和中间件
├── config.py → 集中配置管理
├── database.py → 异步 MySQL 连接池
├── auth.py   → JWT 认证 + 频率限制
├── models.py → Pydantic 数据模型
├── routes_auth.py → 登录/注册/忘记密码
├── routes_projects.py → 项目管理
├── routes_files.py → 文件下载
├── routes_stats.py → 统计排名
└── skills/   → 技能插件系统（可扩展）

运行方式:
python main.py
或: uvicorn python_app.main:app --host 0.0.0.0 --port 3001
=====
"""

import sys
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

# 将父目录加入路径，方便外部 uvicorn 直接运行
sys.path.insert(0, str(__import__("pathlib").Path(__file__).parent.parent))

from .config import JWT_SECRET, DB_CONFIG
from .database import init_db, close_db
from .auth import rate_limit_middleware

# ----- 应用生命周期 -----

@asynccontextmanager
async def lifespan(app: FastAPI):
    """
    应用生命周期管理。

    startup: 初始化数据库连接池 → 加载技能插件
    shutdown: 关闭数据库连接池 → 清理资源
    """
    # === 启动 ===
    print("[启动] 连接数据库...")
    await init_db()
    print("[启动] 数据库连接就绪")

    ... (省略后续代码)
```


14 API 接口速查表

方法	路径	认证	说明
POST	/api/auth/login	否	用户登录 → 返回 JWT Token
POST	/api/auth/register	否	用户注册 → 自动校验参数
POST	/api/auth/forgot-password	否	忘记密码验证身份
GET	/api/auth/me	是	获取当前登录用户信息
GET	/api/projects	是	获取当前用户的项目列表
POST	/api/projects	是	创建新项目 (opencode 会话)
GET	/api/projects/{id}/files	是	获取项目生成文件列表
GET	/api/download/file?path=	是	下载单个文件 (路径白名单)
GET	/api/download/project/{id}	是	按项目打包下载 (tar.gz)
GET	/api/download/latest	是	打包下载最新生成文件
GET	/api/stats/ranking	是	项目 Token 消耗排名
GET	/api/stats/users	是	用户 Token 排名 (含隐私)
GET	/api/skills/hello	否	技能插件示例接口



15 安全特性清单

#	特性	实现方式
1	JWT 密钥保护	环境变量 JWT_SECRET，随机 64 字节
2	路径遍历防护	ALLOWED_DIRS 白名单校验路径
3	命令注入防护	subprocess.run 参数数组传参
4	数据库密码保护	环境变量 DB_PASSWORD
5	错误信息不泄露	统一返回 '服务器内部错误'
6	频率限制	登录 10次/分 · 注册 5次/分
7	密码 bcrypt 哈希	加盐 10 rounds
8	请求体大小限制	最大 100KB 防内存攻击
9	隐私数据遮蔽	非管理员只能看到自己的信息
10	CSRF 防护	Cookie 设置 SameSite=Lax
11	临时文件清理	随机文件名 + 传输后自动删除
12	SQL 注入防护	参数化查询 (%s 占位符)



— End of Document —